Alright, classic **Compiler Design lab** questions 🙂
I'll give you **clean, exam-ready YACC programs** with **Lex + Yacc**, **sample input/output**, and **how it works**. No extra nonsense.

---

1. YACC program to check whether an expression is syntactically correct

Goal

Check whether an arithmetic expression is **valid or invalid**.

Example:
9 + * 3 → ❌ **Invalid Expression**
9 + 3 * 5 → ✅ **Valid Expression**

Lex file – expr.l

```
%{
#include "y.tab.h"
%}

%%
[0-9]+      { return NUM; }
[+\-*/]     { return yytext[0]; }
[\t ]+      ;
\n          { return '\n'; }
.           { return yytext[0]; }
%%

int yywrap() {
    return 1;
}
```

Yacc file – expr.y

```
%{
#include <stdio.h>
#include <stdlib.h>
```

```
int yylex();
void yyerror(const char *s);
%}

%token NUM

%%
stmt    : expr '\n'   { printf("Valid Expression\n"); exit(0); }
        ;

expr    : expr '+' term
        | expr '-' term
        | term
        ;

term    : term '*' factor
        | term '/' factor
        | factor
        ;

factor  : NUM
        | '(' expr ')'
        ;
%%

void yyerror(const char *s) {
    printf("Invalid Expression\n");
    exit(0);
}

int main() {
```

```
    printf("Enter expression: ");
    yyparse();
    return 0;
}
```

Commands to execute

```
lex expr.l
yacc -d expr.y
gcc lex.yy.c y.tab.c -o expr
./expr
```

Sample Output

```
Enter expression: 9 + * 3
Invalid Expression
Enter expression: 9 + 3 * 5
Valid Expression
```

Why this works

- Grammar enforces **operator-operand order**
- Any syntax error → yyerror() called
- So + * without operand is rejected

---

2. Infix to Postfix Converter using YACC

Lex file – infix.l

```
%{
#include "y.tab.h"
%}

%%
[0-9]       { yylval = yytext[0]; return NUM; }
```

```
[+\-*/()]    { return yytext[0]; }
[\t ]+       ;
\n           { return '\n'; }
%%
int yywrap() { return 1; }
```

Yacc file – infix.y

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

%token NUM

%%
stmt    : expr '\n' { printf("\n"); exit(0); }
        ;

expr    : expr '+' term   { printf("+ "); }
        | expr '-' term   { printf("- "); }
        | term
        ;

term    : term '*' factor { printf("* "); }
        | term '/' factor { printf("/ "); }
        | factor
        ;

factor  : '(' expr ')'
        | NUM            { printf("%c ", $1); }
        ;
```

```
%%

int main() {
    printf("Enter infix expression: ");
    yyparse();
    return 0;
}

int yyerror(char *s) {
    printf("Invalid Expression\n");
    return 0;
}
```

Commands to execute

```
lex infix.l
yacc -d infix.y
gcc lex.yy.c y.tab.c -o infix
./infix
```

Sample Input

Enter infix expression: 9+3*5

Output

9 3 5 * +

Why postfix works here

- Grammar enforces **precedence**
- Print operator **after operands**
- So postfix is generated naturally

Viva-ready points (important 🔥)

- YACC checks **syntax**, not semantics

- yyerror() handles invalid expressions
- Left recursion maintains **operator precedence**
- Postfix removes need for parentheses
- Lex does tokenization, Yacc does parsing

If you want:

- **Single combined program**
- **Error recovery**
- **Support multi-digit numbers**
- **Step-by-step parse tree explanation**

Just say the word 👍